

Software Protection: Security Vulnerability Detection

Shuai Wang



香港科技大學

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Some of the slides are written by Suman Jana.

How to find vulnerabilities in software?

```
#include <stdio.h>
#define MAX_IP_LENGTH 15
int main(void) {
    char file_name[] = "ip.txt";
    FILE *fp;
    fp = fopen(file_name, "r");
    char ch;
    int counter = 0;
    char buf[MAX_IP_LENGTH];
    while((ch = fgetc(fp)) != EOF) {
        buf[counter++] = ch;
    }
}
```

Buffer overflow if "ip.txt" has more than 15 bytes.



**BUFFER
OVERFLOW
ATTACKS**

Vulnerability Finding Today

- Security bugs can bring \$500-\$100,000 on the open market
- Good bug finders make \$180-\$250/hr consulting
- Few companies can find good people, many don't even realize this is possible.
 - Google: Team Zero; Tencent: Keen Lab; ...
- Still largely a black art



Security Vulnerabilities

- What can an attacker do with Security bugs?
 - avoid authentication
 - privilege escalation
 - bypass security check
 - deny service
 - run code remotely
- Basically whatever you want.

Automatic Vulnerability Detection



Find a needle in a haystack

Automatic Vulnerability Detection

- Fuzz testing ← discuss today.
- Taint analysis (information flow)
- Concolic execution ← *more comprehensive than testing*
- Symbolic execution ← *more comprehensive than testing*
- Type system ← brief
- Formal verification ← brief
- ...



Formal Verification

- Formal verification can (ideally) **completely eliminate vulnerabilities**.
 - Mathematically prove the absence of bugs.
- How to I know the **insertion sort** will always return a sorted list?

```
# Python program for implementation of Insertion Sort
```

```
# Function to do insertion sort
```

```
def insertionSort(arr):
```

```
    # Traverse through 1 to len(arr)
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        # Move elements of arr[0..i-1], that are  
        # greater than key, to one position ahead  
        # of their current position
```

```
        j = i-1
```

```
        while j >= 0 and key < arr[j] :
```

```
            arr[j+1] = arr[j]
```

```
            j -= 1
```

```
        arr[j+1] = key
```

```
# Driver code to test above
```

```
arr = [12, 11, 13, 5, 6]
```

```
insertionSort(arr)
```

```
print ("Sorted array is:")
```

```
for i in range(len(arr)):
```

```
    print ("%d" %arr[i])
```


Formal Verification

- You can **prove** it, as how you prove some **Euclidean geometry** properties.

```
# Python program for implementation of Insertion Sort
```

```
# Function to do insertion sort
```

```
def insertionSort(arr):
```

```
    # Traverse through 1 to len(arr)
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        # Move elements of arr[0..i-1], that are  
        # greater than key, to one position ahead  
        # of their current position  
        j = i-1
```

```
        while j >= 0 and key < arr[j] :  
            arr[j+1] = arr[j]  
            j -= 1  
        arr[j+1] = key
```

```
# Driver code to test above
```

```
arr = [12, 11, 13, 5, 6]
```

```
insertionSort(arr)
```

```
print ("Sorted array is:")
```

```
for i in range(len(arr)):
```

```
    print ("%d" %arr[i])
```

Proof



```
1 Require Import List.
2 Require Import Zarith.
3 Require Import Bool.
4 Require Import sort_lectures.
5
6 (* Inserts an element in to a sorted list *)
7
8 Function insert (x : Z) (list : list Z) :=
9   match list with
10   | nil => x :: nil
11   | hd :: tl => if (x <= ? hd) then x :: hd :: tl else hd :: insert x tl
12   end.
13
14 (* Insertion sort *)
15 Function insertionSort (list : list Z) :=
16   match list with
17   | nil => nil
18   | hd :: tl => insert hd (insertionSort tl)
19   end.
20
21 (* Insertion of an element keeps the list sorted *)
22 Lemma insert_keeps_list_sorted (x : Z) (list : list Z) :
23   urejen list => urejen (insert x list).
24 Proof.
25   intros.
26   induction list.
27   - auto.
28   - case_eq (x <= ? a) with
29     | yes =>
30       + simpl.
31       + intro.
32       + firstorder.
33       + now apply Z.leb_le.
34     | no =>
35       + apply Z.leb_gt in H0.
36       + simpl.
37       + destruct list.
38       + firstorder.
39       + case_eq (x <= ? y) with
40         | yes =>
41           + firstorder.
42           + now apply Z.leb_le.
43         | no =>
44           + replace (x :: insert x list) with (insert x (x :: list)).
45           + simpl.
46           + case_eq (x <= ? y) with
47             | yes =>
48               + simpl.
49               + intro.
50               + absurd (x <= ? y) with (H0 : false).
51               + auto.
52             | no =>
53               + now rewrite not_false_iff in H0.
54               + auto.
55   Qed.
56
57 (* InsertionSort always returns sorted list *)
58 Lemma insertionSort_returns_sorted_list :
59   forall l : list Z, urejen (insertionSort l).
60 Proof.
61   intros.
62   induction l.
63   - auto.
64   - simpl.
65   - now apply insert_keeps_list_sorted.
66   Qed.
67
68 (* Number of occurrences of x increases if we insert another x into a list *)
69 Lemma occurrences_of_x_increases (x : Z) (l : list Z) :
70   forall l, insert x l = S (count x l).
71 Proof.
72   induction l.
73   - simpl.
74   - now rewrite Z.eqb_refl.
75   - case_eq (x <= ? a) with
76     | yes =>
77       + simpl.
78       + intro.
79       + case_eq (x <= ? a) with
80         | yes =>
81           + intro.
82           + replace a with x.
83           + simpl.
84           + replace (x <= ? a) with true.
85           + auto.
86           + now rewrite Z.eqb_refl.
87         | no =>
88           + auto.
89           + now apply Z.eqb_eq.
90       + intro.
91       + simpl.
92       + replace (x <= ? a) with false.
93       + replace (x <= ? a) with true.
94       + auto.
95       + now rewrite Z.eqb_refl.
96     | no =>
97       + intro.
98       + case_eq (x <= ? a) with
99         | yes =>
100           + intro.
101           + replace a with x.
102           + simpl.
103           + replace (x <= ? a) with true.
104           + auto.
105           + now rewrite Z.eqb_refl.
106         | no =>
107           + intro.
108           + simpl.
109           + now replace (x <= ? a) with false.
110   Qed.
111
112 (* Number of occurrences of x does not change if we insert a different element into *)
113 Lemma occurrences_of_x_does_not_change (x : Z) (y : Z) (l : list Z) :
114   ((x <= ? y) = false) => forall x (insert y l) = forall x l.
115 Proof.
116   induction l.
117   - simpl.
118   - now replace (x <= ? y) with false.
119   - simpl.
120   - case_eq (y <= ? a) with
121     | yes =>
122       + intro.
123       + case_eq (x <= ? a) with
124         | yes =>
125           + intro.
126           + simpl.
127           + now replace (x <= ? a) with false.
128         | no =>
129           + intro.
130           + simpl.
131           + now replace (x <= ? a) with false.
132   Qed.
```

Computer will check the correctness

Haven't finished yet..

Formal Verification

- Two important elements in a formal verification:
 - **Specification** ← what properties I would like to prove?
 - Sorting? Or memory safety?
 - **Inductive proofs**
 - You write your proof as code, and computers will check the correctness of your proof.
 - **Difficult** to manually write the proof, but **relatively easy** to verify a given proof.

But no solution is perfect! Even formal verification

- But impractical in general...
 - Formal verification is hard in general, impossible for big things.
 - Take a Ph.D. several years, write million lines (?) of proof code, to verify (simple) memory safety properties of a tiny OS kernel component
 - $2 \sim 5 \times N$ lines of proof for N lines of code, or even worse.
- Adoption of formal verification in the industry

Applied Scientist Intern, Automated Reasoning - Amazon Web Services

Job ID: 943766 | Amazon Web Services, Inc.

DESCRIPTION

The Automated Reasoning Group in AWS Platform is looking for PhD Interns interested in formal methods and programming languages. Using automated reasoning technology and mathematical proofs, AWS allows customers to answer questions about security, availability, durability, and functional correctness. We call this provable security, absolute assurance in security of the cloud and in the cloud. <https://aws.amazon.com/security/provable-security/>

Type System

- Type system (safe language)
 - Provide strong guarantee on well-typed programs
 - Suppose we have two variables N and M in our financial trading software.
 - $N \times M \leftarrow$ seems all right
 - $N \times M \leftarrow$ but not correct if N's type is US Dollars and M's type is HK Dollars
 - this doesn't eliminate all problems, although with such "annotations" we can quickly pinpoint some defects.
 - But type check is usually super fast, and does not require much extra cost.
 - Performance, existing code base, flexibility, programmer capability, etc.
 - Financial trading companies are major sponsors of the safe language communities
 - Java; C#; Scala; OCaml; Haskell
 - Jane Street; Goldman Sachs

Automatic Vulnerability Detection

- Fuzz testing ← discuss today.
- Taint analysis (information flow) ← discuss today
- Concolic execution ← *more comprehensive than testing*
- Symbolic execution ← *more comprehensive than testing*
- Type system
- Formal verification
- ...



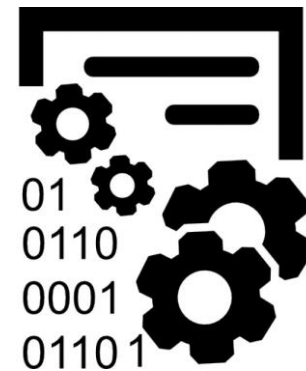
Fuzz Testing

A **simple** but very well-performing **testing method** to find **vulnerabilities**.

- Automatically generate test cases
- Many abnormal/random/corner-case test cases are input into a target
- Application is monitored for errors



Random
input
→



Test program

Fuzz Testing

- Given a program simply feed **random** inputs **to execute** and see whether it trigger errors (**usually “crash”; see the next slide**)
 - **To execute:**
 - We find a bug and we know how to trigger it at the same time
 - Why is this so critical?
 - Very likely we (attacker) can **control** the bug!
 - **Random:**
 - We find many corner cases by using **invalid inputs**
 - Invalid inputs: initial design goal, but extended with “grammar-aware” fuzzing later.
 - In contrast, many of the **software testing techniques** assert **correct functionality** with **well-formed inputs**.

Fuzz Testing

- Given a program simply feed **random** inputs **to execute** and see whether it trigger errors
 - Errors
 - **Crash**: highly like indicates **memory access errors** and therefore cause memory overread/overwrite vulnerabilities
 - **Hang**: system lost response (not “available” anymore)
 - **Race condition**: need to enable **dynamic error detector**...
 - **Other errors**: user-after free; double-free (*introduce later this semester*)
 - More stealthy than direct crash
 - Run program under **dynamic memory error detector**
 - But fuzz testing is usually not used to find **logic errors**.

Software Testing vs. Fuzzing

A very high-level comparison between software testing (e.g., regression testing) and fuzz testing.

	Software Testing	Fuzzing
Definition	Run program on many normal inputs , look for violation of testing oracle	Run program on many abnormal inputs , look for badness
Goals	Prevent normal users from encountering errors	Prevent attackers from identifying exploitable errors

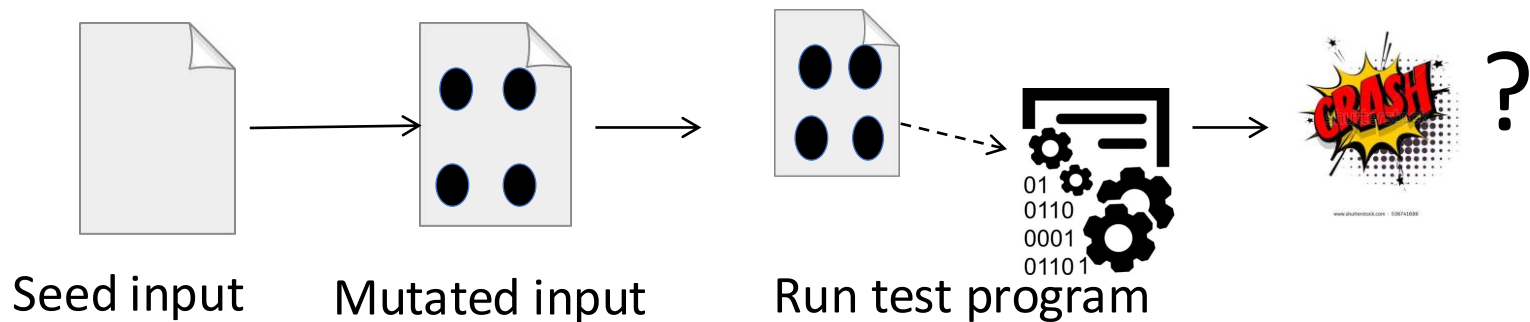
Fuzz Testing Techniques

Mutation strategies	Knowledge of inputs	Knowledge of the fuzzed software	
		Black Box (old)	Gray box (State of the art)
	Random-mutation	Easy but ineffective	Industry Strength
	Generation-based mutation	Not bad	Industry Strength

However, it needs input specifications which are often not available.

Mutation-Based fuzzing

- Take a ~~well-formed~~ well-formed or random inputs, **randomly perturb** (flipping bit, etc.)
 - although “well-formed” inputs are generally more desired and empirically better.
- Little or no knowledge of **the structure of the inputs** is assumed
- Anomalies are added to existing inputs
 - Anomalies may be completely random or follow some heuristics (e.g., remove NULL, shift character forward)
- Examples: ZZUF, Taof, GPF, ProxyFuzz, FileFuzz, Filep, etc.



Example: fuzzing a PDF viewer

- Google for .pdf (about 1 billion results)
- Crawl pdf files.
- Use fuzzing tool (or script)
 - Collect seed PDF files
 - Randomly mutate a file
 - Feed it to the program
 - Record if it crashed (and input that crashed it)

Mutation-based fuzzing

- Super easy to setup and automate
- Little or no input format knowledge is required
- May fail at the early stage:
 - Parsing; format checking; checksum; ...
 - Good or bad?

Fuzz Testing Techniques

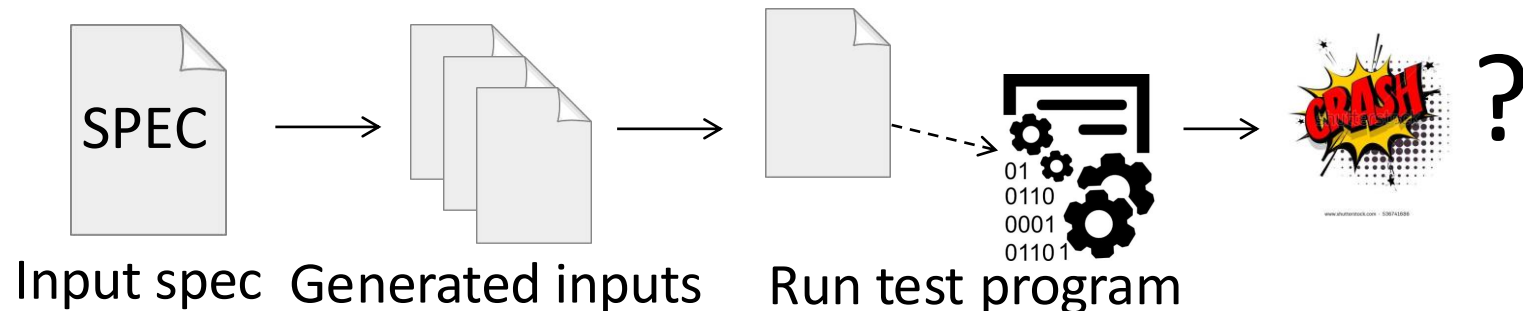
Mutation strategies

	Black Box (old)	Gray box (State of the art)
Random-mutation	Easy but ineffective	Industry Strength
Generation-based mutation	Not bad	Industry Strength

However, it needs input specifications which are often not available.

Generation-Based (Grammar-Aware) Fuzzing

- Test cases are generated from **some description of the input format**: documentation, etc.
 - Using specified protocols/file format info
- Pre-knowledge of protocol/format should give better results than random fuzzing
 - By being able to directly generate many structured inputs
 - Often work together with random mutation to explore different components.

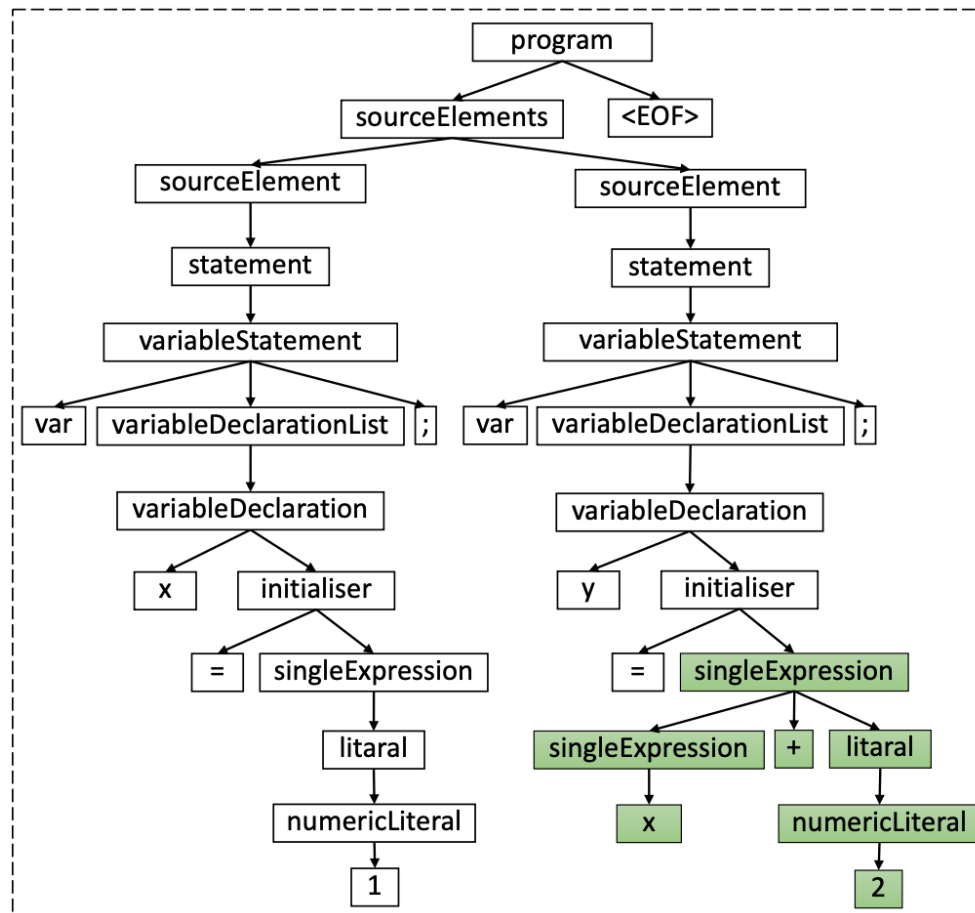


Generation-Based Fuzzing

```
EXPR_GRAMMAR = {  
    "<start>":  
        ["<expr>"],  
  
    "<expr>":  
        ["<term> + <expr>", "<term> - <expr>", "<term>"],  
  
    "<term>":  
        ["<factor> * <term>", "<factor> / <term>", "<factor>"],  
  
    "<factor>":  
        ["+<factor>",  
         "-<factor>",  
         "(<expr>)",  
         "<integer>.<integer>",  
         "<integer>"],  
  
    "<integer>":  
        ["<digit><integer>", "<digit>"],  
  
    "<digit>":  
        ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]  
}
```

Grammar of a simple calculator

Structure-Based Fuzzers



How to fuzz Python/JavaScript Interpreter?

--> tree structure mutation of syntax valid programs

But grammars could be very complex

60 / 774

4.2 Primitive Types and Values

A primitive type is predefined by the Java programming language and named by its reserved keyword (§3.9):

PrimitiveType:
 {Annotation} NumericType
 {Annotation} boolean

NumericType:
 IntegralType
 FloatingPointType

Java Specification has 774 pages...

It could be difficult to define good generation rules for real-world cases..

➔ State-of-the-art: use deep learning (language models) to “learn” this knowledge...

Mutation-based vs. Generation-based

- Mutation-based fuzzer
 - **Pros:** Easy to set up and automate, little to no knowledge of input format required
 - **Cons:** Limited by initial input seeds, may fail for protocols with checksums and other hard checks
- Generation-based fuzzers
 - Pros: completeness, theoretically can deal with complex dependencies
 - Cons: writing generators is hard, performance depends on the quality of the SPEC → **very often difficult to get SPEC**
- Luckily, “structure/grammar” information could be partially **inferred on the fly!!!**
 - With some **genetic algorithm** → grey box fuzzing.

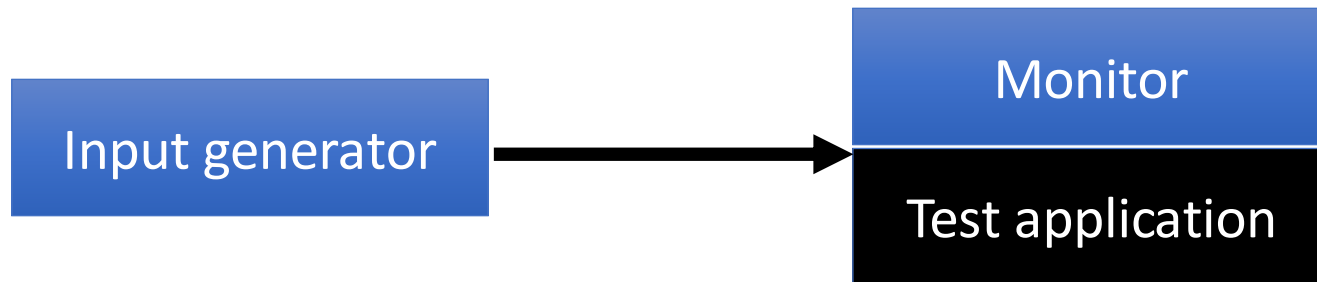
Fuzz Testing Techniques

Mutation strategies	Knowledge of inputs	Knowledge of the fuzzed software	
		Black Box (old)	Gray box (State of the art)
	Random-mutation	Easy but ineffective	Industry Strength
	Generation-based mutation	Not bad	Industry Strength

However, it needs input specifications which are often not available.

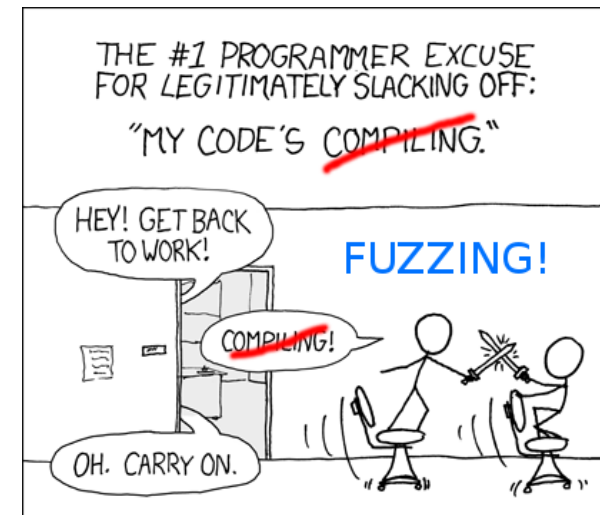
Blackbox Fuzzing

- Keep generating mutated inputs and feed to the test application.
- Application is monitored for errors



Pros/Cons of Blackbox Fuzzing

- Advantage: easy, low programmer cost
 - Well, every “script kid” know blackbox fuzzing from day one...
- Disadvantage: inefficient (particularly for **random mutation**)
 - Do your best and need some luck
 - Inputs often require structures, random inputs are likely to be malformed and **rejected at the early stage**
 - Inputs that trigger an incorrect behavior is a **very small fraction**, probably of getting lucky is **very low**



How much fuzzing is enough?

- Fuzzer may generate an infinite number of test cases. **When has the fuzzer run long enough?**
 - Code coverage is used as the “feedback” to guide fuzzer.
- **Code coverage** is a metric that can be used to determine how much code **has been executed**.
 - Data can be obtained using a variety of profiling tools.
e.g. gcov, lcov
 - Line coverage; branch coverage; path coverage;

Line coverage

- **Line/block coverage:** Measures how many lines of source code have been executed.
- For the code on the right, how many test cases (values of pair (a,b)) needed for full (100%) line coverage?

```
if( a > 2 )  
    a = 2;  
if( b > 2 )  
    b = 2;
```


Branch coverage

- **Branch coverage:** Measures how many branches in code have been taken (conditional jumps)
- For the code on the right, how many test cases needed for full branch coverage?

```
if( a > 2 )  
    a = 2;  
if( b > 2 )  
    b = 2;
```

Path coverage

- **Path coverage:** Measures how many paths have been taken
- For the code on the right, how many test cases needed for full path coverage?

```
if( a > 2 )  
    a = 2;  
if( b > 2 )  
    b = 2;
```

Benefits of Code coverage

- Can answer the following questions

- How good is an initial file?

- Am I getting stuck somewhere?

```
if (packet[0x10] < 7) {  
    //hot path  
} else {  
    //cold path  
}
```

- How good is fuzzerX vs. fuzzerY

- Am I getting benefits by running multiple fuzzers?

Fuzz Testing Techniques

Mutation strategies

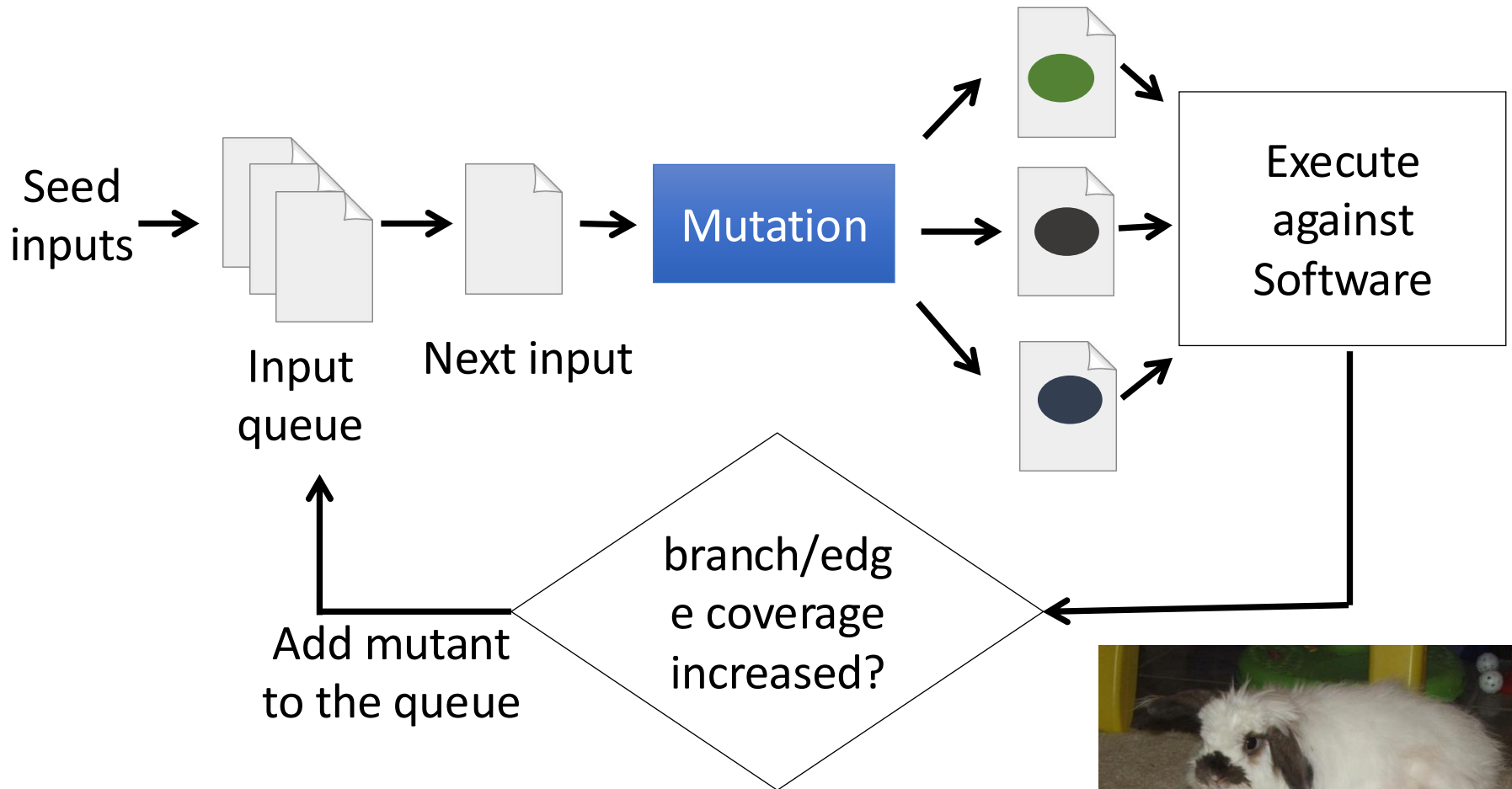
	Black Box (old)	Gray box (State of the art)
Random-mutation	Easy but ineffective	Industry Strength
Generation-based mutation	Not bad	Industry Strength

However, it needs input specifications which are often not available.

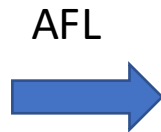
Coverage-guided GrayBox Fuzzing

- The state of the art
 - Run mutated inputs and measure code coverage
 - Search for mutants that result in coverage increase
 - Often use (conceptually) a “genetic algorithm”, i.e., try random mutations on test corpus and only keep mutants for further usage if coverage increases
 - Examples: AFL, libfuzzer

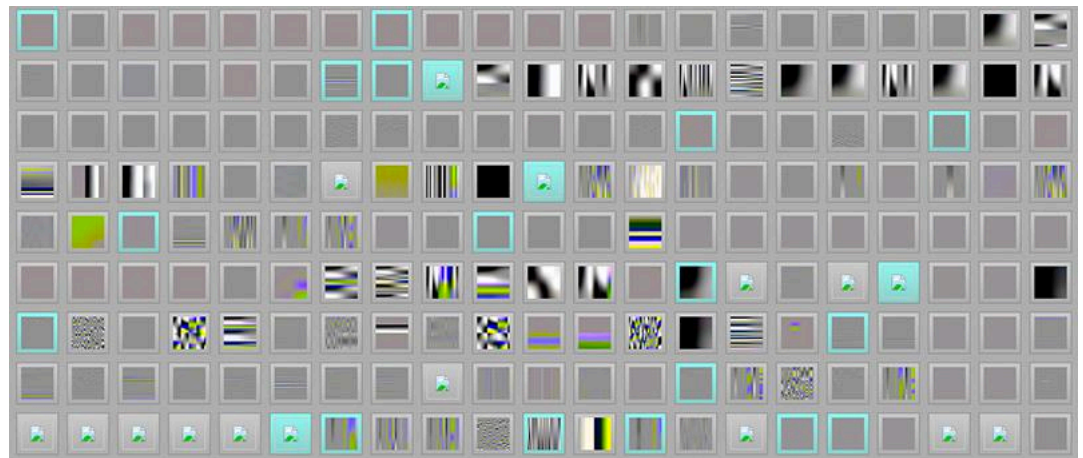
American Fuzzy Lop (AFL)



The powerful of genetic algorithm



Just random junk data...



Random images but with **legit JPEG format!!!**

Good coverage-guided fuzzer can gradually “synthesize” well-formed input format to certain degree, by keeping mutants with new coverage and proceed further.

The input with relatively more legitimate formats got more trials!

Fuzzing challenges

- How to seed a fuzzer?
 - (Ideally) seed inputs must cover different branches
 - Remove duplicate seeds covering the same branches
- Some branches might be very hard to get past as the # of inputs satisfying the conditions are very small
 - Manually/automatically transform/remove those branches

Hard to fuzz code

```
void test (int n) {  
    if (n==0x12345678)  
        crash();  
}
```

needs 2^{32} attempts
In the worst case

Make it easier to fuzz

```
void test (int n) {  
    int dummy = 0;  
    char *p = (char *) &n;  
    if (p[3]==0x12) dummy++;  
    if (p[2]==0x34) dummy++;  
    if (p[1]==0x56) dummy++;  
    if (p[0]==0x56) dummy++;  
    if (dummy==4)  
        crash();  
}
```

The time complexity is largely reduced from 2^{32} to roughly $(2^8 + 2^8 + 2^8 + 2^8)$.

Fuzzing rules of thumb

- **Input-format knowledge** is very helpful
- Beat random with **feedback guide**, and better specs make better fuzzers
- Time matters
 - More fuzzing is obviously better
 - It is recommended to fuzz at least 12 hours for real-world applications.
- Best results come from guiding the process

From Traditional Fuzzing to AI-Enhanced Fuzzing

"Can AI help us fuzz smarter?"

- Traditional fuzzing relies on random or coverage-guided mutation
- AI/ML/LLM techniques are being integrated into every stage of fuzzing
- This section: how and where AI improves fuzzing

Challenges in Traditional Fuzzing

- Structured inputs: PDF, JSON, SQL, Protocol Buffers
— random bytes are rejected by parsers immediately
- Seed explosion: Seed queue keeps growing, scheduling becomes inefficient
- Blind mutation: No knowledge of "which bytes" affect "which branch"
- Harness writing cost: Manually writing a fuzz driver for each target function is expensive

Overview: Where Does AI Fit in Fuzzing?

Fuzzing Stage	AI Technique	Representative Work
Input generation / mutation	RNN / Transformer / LLM	Learn&Fuzz, Neuzz, Titanfuzz, ...
Seed scheduling	Reinforcement Learning	EcoFuzz
Harness generation	LLM (Code Generation)	OSS-Fuzz-Gen

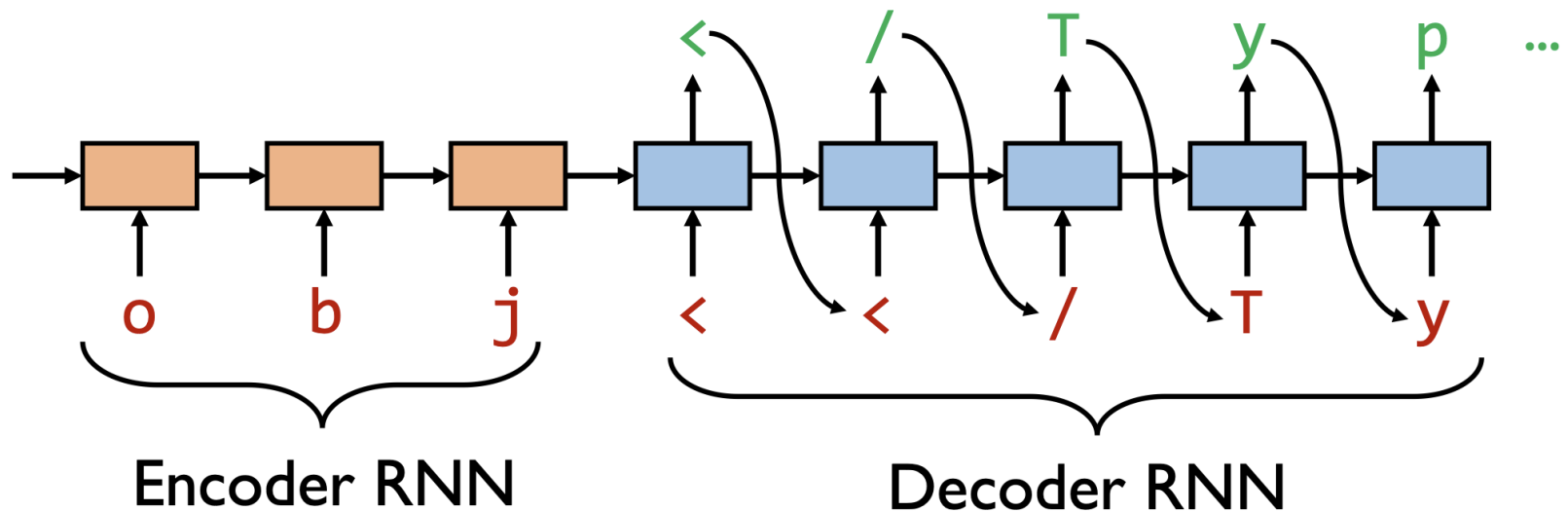
Track 1: Neural Network–Based Input Generation

- Core Idea:
 - Train a neural network to learn the distribution $P(x)$ of valid inputs
 - Sample from the learned distribution to produce new, slightly abnormal test inputs
- Advantages:
 - Can generate highly structured inputs (satisfying grammar)
 - Introduces "mutations" at the boundary → more likely to trigger bugs
- Applicable to:
 - PDF, JavaScript, HTML, network protocol messages, etc.

Case Study 1: Learn&Fuzz (ASE 2017)

- Target:
 - Automatically generate valid but tricky PDF objects
- Method:
 - Collect a large corpus of valid PDF files
 - Train a character-level LSTM on PDF objects
 - At sampling time, vary temperature τ :
 - Small $\tau \rightarrow$ close to valid input
 - Large $\tau \rightarrow$ more "mutated"
 - Use sample-random strategy: with probability p sample from model, with probability $1-p$ pick a random character

Case Study 1: Learn&Fuzz (ASE 2017)

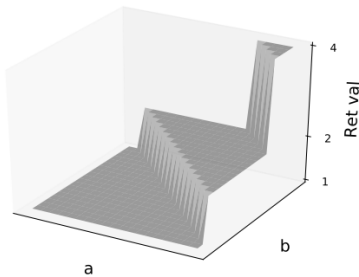


A sequence-to-sequence RNN model to generate PDF objects.

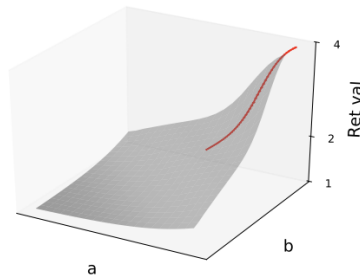
Case Study 2: Neuzz (S&P 2019)

- Key Observation: Branch coverage is a function of input $f: \{0,1\}^n \rightarrow \{0,1\}^m$, but it is discrete and non-differentiable
- Method:
 - Train a feedforward neural network to approximate $f_{NN}(x) \approx f(x)$
 - Compute gradient $\nabla_x f_{NN}(x)$ with respect to input bytes
 - Mutate seed bytes along the gradient direction
- Key Intuition:
 - Gradient tells us which bytes to change and in what direction
 - Replaces blind random mutation with targeted, gradient-guided mutation

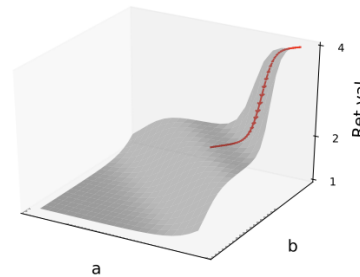
Case Study 2: Neuzz (S&P 2019)



(a) Original



(b) NN smoothing



(c) NN smoothing + refining

```
1 z = pow(3, a+b);  
2 if(z < 1) {  
3   return 1;  
4 }  
5 else if(z < 2) {  
6   //vulnerability  
7   return 2;  
8 }  
9 else if(z < 4) {  
10  return 4;  
11 }
```

Simple code snippet demonstrating the benefits of neural smoothing for fuzzing.

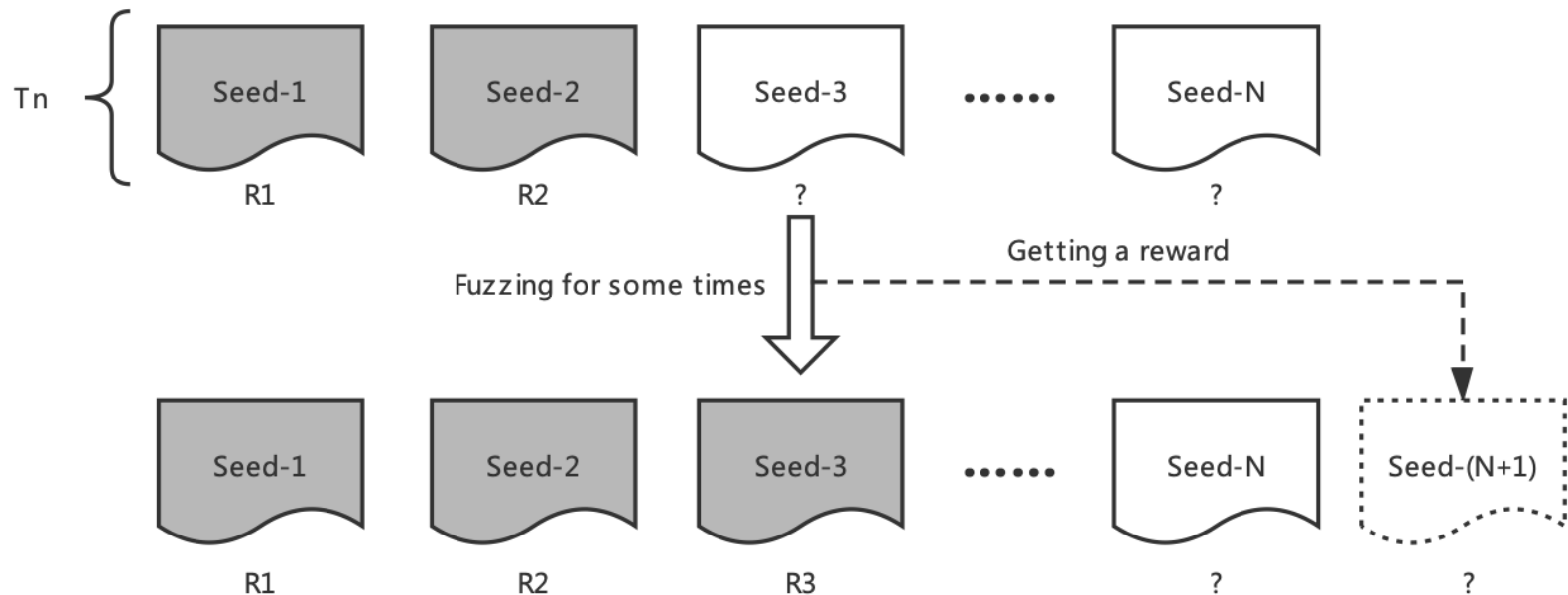
Track 2: Reinforcement Learning for Seed Scheduling

- Formulation as an RL Problem:
 - State s : Current coverage bitmap, seed queue status, remaining time budget
 - Action a : Which seed to pick / which mutation operator to apply
 - Reward r : Whether new coverage or new crash is found

Case Study 3: EcoFuzz (USENIX Sec 2020)

- Problem: AFL's power schedule is static; wastes time on "low-value" seeds
- Approach:
 - Model seed scheduling as a Variant of the Adversarial Multi-Armed Bandit (VAMAB)
 - Each seed = one arm
 - Reward of each trial = generating an input that triggers new path
 - VAMAB is a classical concept from Multi-Armed Bandit (MAB), and explained the trade-off between exploration and exploitation in Coverage-based Greybox Fuzzing (CFG)

Case Study 3: EcoFuzz (USENIX Security 2020)



The figure illustrating VAMAB model, in which the grey color block symbolizes that this seed has been fuzzed.

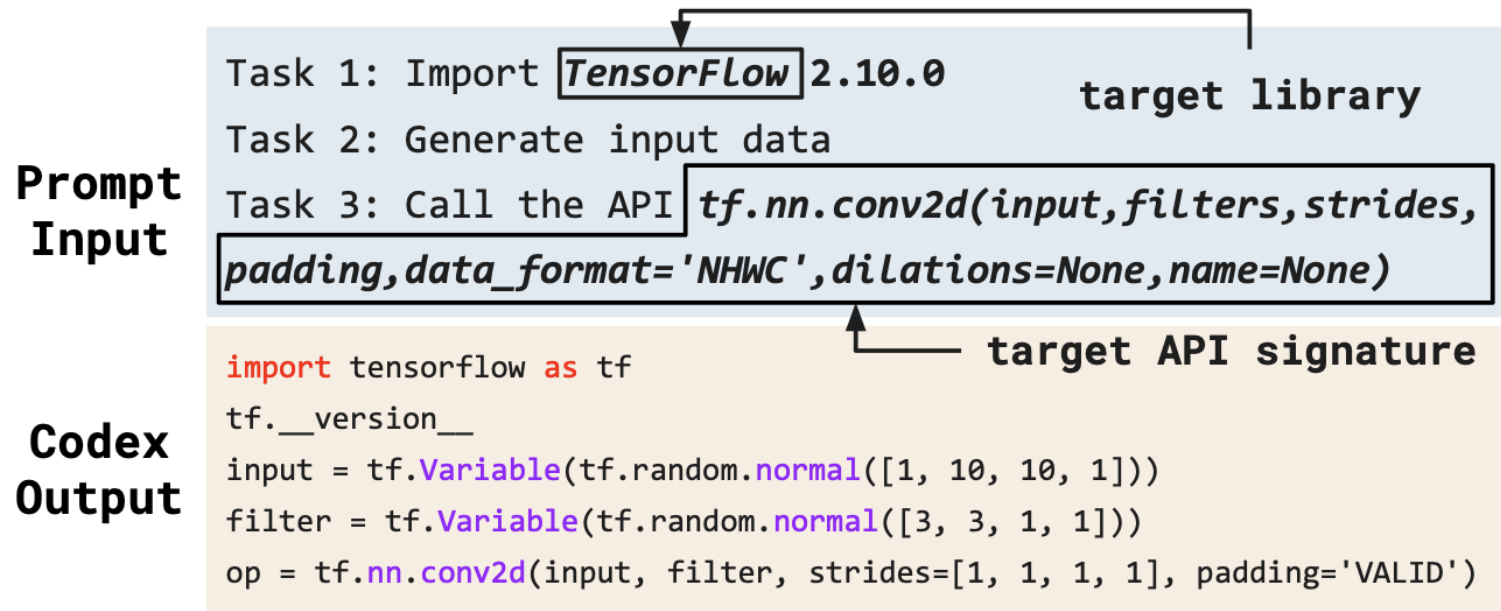
Track 3: Large Language Models Meet Fuzzing

- Why are LLMs suitable for fuzzing?
 - Pre-trained on massive code and structured data corpora
 - Natively "understand" many input grammars: JSON, XML, SQL, C, JavaScript...
 - Can be flexibly guided via prompt engineering
- Three major application directions:
 - Generate structured test inputs
 - Automatically write fuzz harnesses
 - Analyze and triage discovered bugs

Case Study 4: TitanFuzz (ISSTA 2023)

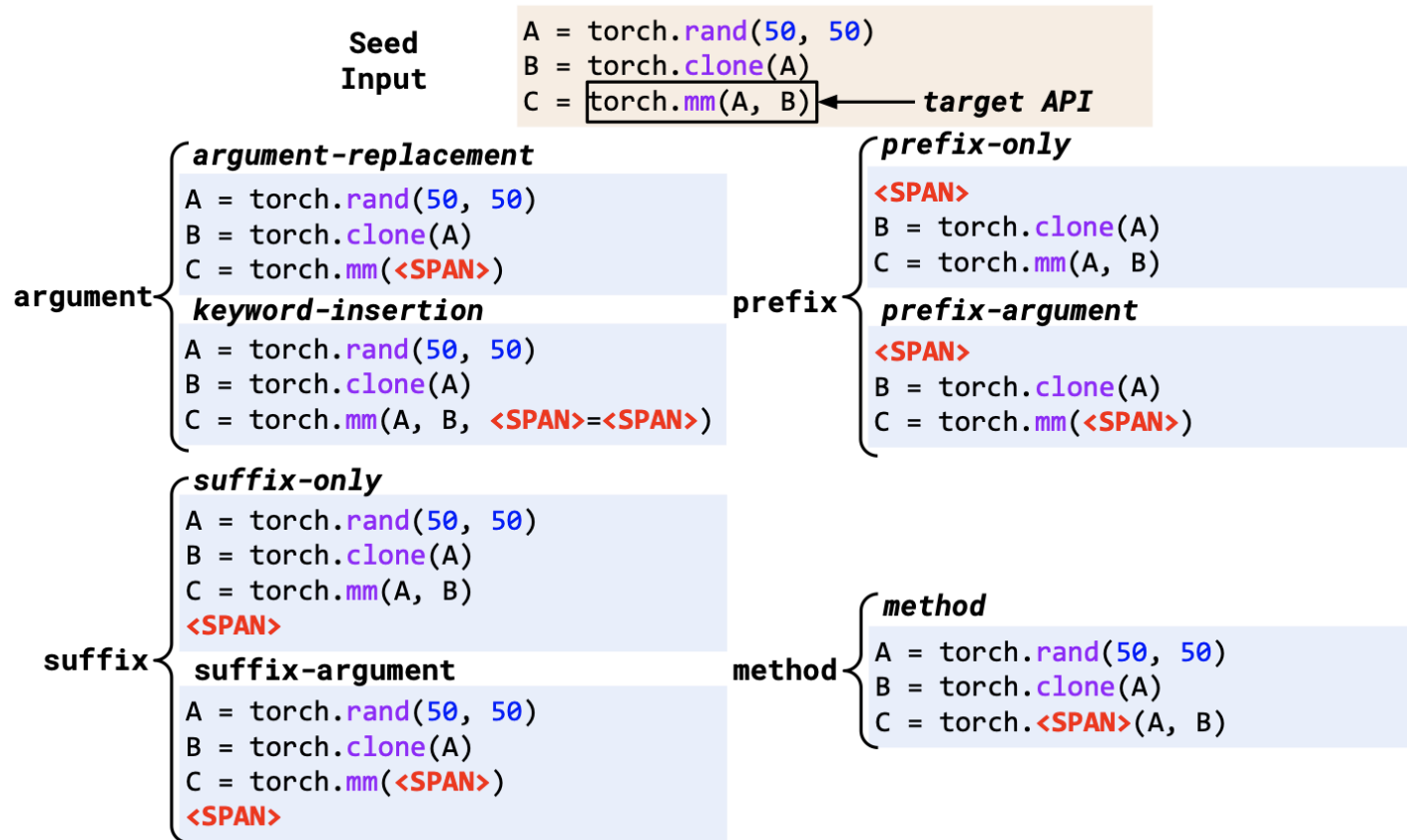
- Target: Fuzz PyTorch / TensorFlow APIs
- Challenge: DL API calls must satisfy complex type and semantic constraints
- Method:
 - Generation phase: Use Codex / InCoder's infilling capability to generate API call sequences
 - Mutation phase: Use LLM to perform mask-and-infill mutation on existing code

Case Study 4: TitanFuzz (ISSTA 2023)



Example generation from the Codex model.

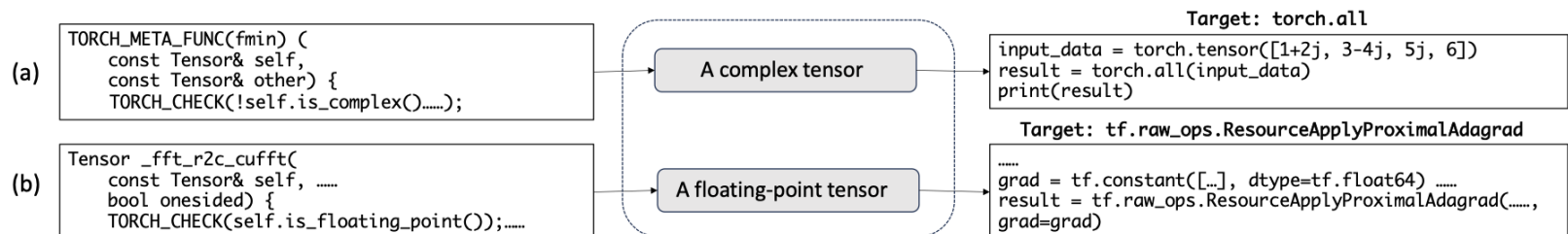
Case Study 4: TitanFuzz (ISSTA 2023)



Mutation operators outputs (inputs for the model).

Case Study 5: DFUZZ (ICSE 2024)

- Challenge:
 - DL libraries contain thousands of APIs with complex input constraints
 - Existing fuzzers (e.g., TitanFuzz) generate inputs blindly — no knowledge of what patterns actually cause bugs
 - Many APIs share similar logic → same bug pattern recurs across APIs
- Key Insight: Bug-fix commits are a goldmine — each fix reveals an exploit pattern



Case Study 6: G2FUZZ (USENIX 2025)

- Target: Programs processing non-textual inputs (images, audio, video, fonts, ELF binaries...)
- Challenge:
 - LLMs operate on tokens, cannot directly generate valid binary/multimedia files
 - Querying LLM per input is expensive and slow (~2 queries/s vs AFL's ~5000 exec/s)
 - Traditional grammar-based generators require weeks of manual specification
- Key Insight: Generate the generator, not the input
 - Existing LLM fuzzing: LLM → 1 test input (repeat for each input)
 - G2FUZZ: LLM → generator.py → several test inputs (one-time LLM cost)

```
1 from PIL import Image
2 image = Image.new('RGB', (100, 100))
3 image.save('./tmp/tiff.tiff',
             compression='tiff_lzw')
```

Python generator for creating the TIFF file with LZW compression data.

Future Directions

- End-to-end AI Fuzzer:
 - From harness generation → input generation → bug analysis — full pipeline
- Small model distillation:
 - Replace GPT-4 API with local lightweight models to reduce cost
- Multi-modal fuzzing:
 - Combine code + binary + documentation understanding
- AI Agents for bug hunting:
 - Autonomous agents that explore, analyze, and verify vulnerabilities